

MyMquid

BACKEND SPEC — NOTIFICATIONS

Blog Event Notification System

Specification for the backend notification system that automatically creates in-app notifications for all active users whenever a blog post is **created**, **updated**, **published**, or **deleted**.

Version 1.0

June 2026

NestJS

Backend Only

⚠ This is a backend-only implementation. The frontend notification UI, badge count, and API calls are already built and working — only the server-side event handling needs to be added.



Overview

What's already built, what the backend needs to add

What Is Already Working (Frontend) DONE

NOTIFICATION BELL (TOPNAV)

Shows a red badge with the unread count. Reads from `useNotificationStore.unreadCount`. Updates immediately after every blog save or delete — no page reload needed.

NOTIFICATIONS PAGE

Lists all notifications from `GET /notifications`. Users can mark individual items or all as read. Badge count decrements accordingly.

AUTO-REFRESH TRIGGER

After every `savePost()` and `deletePost()` the store calls `GET /notifications` silently in the background. If the backend has new records, the badge updates instantly.

What the Backend Needs to Add BACKEND WORK

The frontend already fetches and displays notifications — it just needs the backend to **write notification records to the database** when blog events happen. No new API endpoints are required. This is purely internal server-side logic.

SUMMARY OF WORK REQUIRED

WHAT TO DO	WHERE	DETAILS
Create <code>notifications</code> table	Database migration	If not already created from the existing <code>GET /notifications</code> endpoint
Write a <code>NotificationsService</code>	NestJS service	Method: <code>createForAllUsers(event)</code> — inserts one row per active user
Hook into <code>BlogService</code>	NestJS service	Call <code>NotificationsService.createForAllUsers()</code> after create, update, delete

How the Frontend Refresh Works

- 1

Admin saves or deletes a blog post
Frontend calls `POST /blog`, `PUT /blog/:id`, or `DELETE /blog/:id`
- ↓
- 2

Backend processes the blog request AND writes notification records



3

Frontend blog store receives success response

Shows toast ("Post published!" / "Post deleted.")



4

Frontend silently calls GET /notifications in the background

File: `src/admin/blog/useBlogStore.ts` — triggered automatically after every save/delete



5

Notification store updates — badge count refreshes instantly

The red number on the bell icon in the top navigation updates to reflect new unread notifications



Blog Events & Notification Content

Exactly what to create in the database for each blog action

Events That Trigger Notifications

Blog Action	API Call	Condition	Notification Title	Type
Post Created (Draft)	<code>POST /blog</code>	<code>status === "draft"</code>	"New Draft Created"	info
Post Published	<code>POST /blog</code> or <code>PUT /blog</code>	<code>status === "published"</code>	"Post Published"	success
Post Scheduled	<code>POST /blog</code> or <code>PUT /blog</code>	<code>status === "scheduled"</code>	"Post Scheduled"	info
Post Updated	<code>PUT /blog/:id</code>	Any update (status unchanged)	"Post Updated"	info
Post Deleted	<code>DELETE /blog/:id</code>	Always	"Post Deleted"	warning

Tip: For PUT requests, check the incoming `status` field to decide which event applies. If the status changed from draft to published, emit "Post Published" — not "Post Updated".

Notification Message Examples

Include the post title in the message for context

POST PUBLISHED

```
{
  "title": "Post Published",
  "message": "\"Cloud Security Best Practices\" is now live.",
  "type": "success"
}
```

NEW DRAFT CREATED

```
{
  "title": "New Draft Created",
  "message": "\"AI in Modern Business\" was saved as a draft.",
  "type": "info"
}
```

POST DELETED

```
{
  "title": "Post Deleted",
  "message": "\"2026 Industry Trends\" was permanently deleted."
}
```

POST UPDATED

```
{
  "title": "Post Updated",
  "message": "\"Our Technology Solutions\" was updated by Patrick Evra."
}
```

```

    "type": "warning"
  }

```

```

    "type": "info"
  }

```

⚠ Backend implementation required

Full Notification Record to Insert

One row per active user per event

DATABASE ROW SHAPE

```

{
  id:          uuid (auto-generated),
  user_id:     string,    -- FK → users.id
  title:       string,
  message:     string,
  type:        "info" | "success" |
               "warning" | "error",
  read:        false,    -- always false
                   on creation
  created_at:  now()
}

```

WHAT GET /NOTIFICATIONS RETURNS (PER ROW)

```

{
  "id":        "uuid",
  "title":     "Post Published",
  "message":   "\"Cloud Security...\" is
now live.",
  "type":      "success",
  "read":      false,
  "createdAt": "2026-06-01T12:00:00Z"
}

```

GET /notifications must filter by user_id of the authenticated user — each user only sees their own notifications.



Implementation Guide

NestJS code structure and implementation steps

Step 1 — Create NotificationsService Method

NOTIFICATIONS.SERVICE.TS

```
// Add this method to your existing NotificationsService

async createForAllUsers(payload: {
  title: string;
  message: string;
  type: 'info' | 'success' | 'warning' | 'error';
}): Promise<void> {
  // Fetch all active user IDs
  const users = await this.usersRepository.find({
    where: { active: true },
    select: ['id'],
  });

  // Insert one notification row per user
  const notifications = users.map((user) =>
    this.notificationsRepository.create({
      userId: user.id,
      title: payload.title,
      message: payload.message,
      type: payload.type,
      read: false,
    })
  );

  await this.notificationsRepository.save(notifications);
}
```

Step 2 — Hook into BlogService

BLOG.SERVICE.TS — INJECT NOTIFICATIONSSERVICE

```
// Inject NotificationsService into BlogService constructor
constructor(
  @InjectRepository(BlogPost)
  private readonly blogRepository: Repository<BlogPost>,
  private readonly notificationsService: NotificationsService, // ← add this
) {}
```

AFTER CREATE (POST /BLOG)

```
async create(dto: CreateBlogPostDto, author: AdminUser): Promise<BlogPost> {
  const post = await this.blogRepository.save({ ...dto, author });

  // Notify all users based on status
  const isPublished = post.status === 'published';
  const isScheduled = post.status === 'scheduled';

  await this.notificationsService.createForAllUsers({
    title: isPublished ? 'Post Published' : isScheduled ? 'Post Scheduled' : 'New Draft Created',
    message: isPublished
      ? `"${post.title}" is now live.`
      : isScheduled
      ? `"${post.title}" is scheduled for publication.`
      : `"${post.title}" was saved as a draft.`,
    type: isPublished ? 'success' : 'info',
  });

  return post;
}
```

AFTER UPDATE (PUT /BLOG/:ID)

```
async update(id: string, dto: UpdateBlogPostDto, author: AdminUser): Promise<BlogPost> {
  const existing = await this.findOneOrFail(id);
  const updated = await this.blogRepository.save({ ...existing, ...dto });

  // Detect if status changed to published
  const justPublished = existing.status !== 'published' && updated.status === 'published';

  await this.notificationsService.createForAllUsers({
    title: justPublished ? 'Post Published' : 'Post Updated',
    message: justPublished
      ? `"${updated.title}" is now live.`
      : `"${updated.title}" was updated by ${author.name}.`,
    type: justPublished ? 'success' : 'info',
  });

  return updated;
}
```

AFTER DELETE (DELETE /BLOG/:ID)

```
async delete(id: string): Promise<void> {
  const post = await this.findOneOrFail(id);
  await this.blogRepository.delete(id);

  await this.notificationsService.createForAllUsers({
    title: 'Post Deleted',
    message: `"${post.title}" was permanently deleted.`,
    type: 'warning',
  });
}
```

```
}},  
}
```




Database Schema & Implementation Checklist

Table structure and step-by-step checklist

Notifications Table Schema

SQL (POSTGRESQL)

```
CREATE TABLE notifications (  
  id          UUID PRIMARY KEY DEFAULT  
    gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES  
    users(id) ON DELETE CASCADE,  
  title       VARCHAR(255) NOT NULL,  
  message     TEXT NOT NULL,  
  type        VARCHAR(20) NOT NULL  
    CHECK (type IN  
('info','success','warning','error')),  
  read        BOOLEAN NOT NULL DEFAULT  
    false,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT  
    now()  
);  
  
-- Index for fast per-user queries  
CREATE INDEX idx_notifications_user_id  
  ON notifications(user_id, created_at  
  DESC);
```

TYPEORM ENTITY

```
@Entity('notifications')  
export class Notification {  
  @PrimaryGeneratedColumn('uuid')  
  id: string;  
  
  @ManyToOne(() => User, { onDelete:  
'CASCADE' })  
  @JoinColumn({ name: 'user_id' })  
  user: User;  
  
  @Column()  
  title: string;  
  
  @Column('text')  
  message: string;  
  
  @Column()  
  type: 'info' | 'success' | 'warning' |  
'error';  
  
  @Column({ default: false })  
  read: boolean;  
  
  @CreateDateColumn({ name: 'created_at'  
  })  
  createdAt: Date;  
}
```

GET /NOTIFICATIONS QUERY (FILTER BY LOGGED-IN USER)

```
SELECT id, title, message, type, read, created_at  
FROM   notifications  
WHERE  user_id = :currentUserId  
ORDER  BY created_at DESC  
LIMIT  50;
```

Important Notes

PERFORMANCE

- Use `INSERT ... SELECT` to batch-insert all user rows in a single query instead of a loop
- Add index on `(user_id, created_at DESC)` for fast reads
- Cap `GET /notifications` at 50 most recent — the frontend does not paginate notifications

EDGE CASES

- Wrap notification creation in a `try/catch` — notification failure should NOT cause the blog API to return an error
- Notifications are per-user — deactivated users (`active: false`) should NOT receive notifications
- The author who made the change can receive the notification too (optional, your call)

Critical: Do not let notification creation errors break the blog API response. Always wrap the `createForAllUsers()` call in a `try/catch` with silent failure — the blog operation (create/update/delete) must succeed even if notifications fail.

Implementation Checklist

DATABASE

- ☐ Create `notifications` table (migration)
- ☐ Add `user_id` foreign key → `users.id`
- ☐ Add index on `(user_id, created_at DESC)`

NOTIFICATIONSSERVICE

- ☐ Add `createForAllUsers(payload)` method
- ☐ Filter only `active = true` users
- ☐ Batch insert (not loop) for performance

BLOGSERVICE HOOKS

- ☐ Inject `NotificationsService` into `BlogService`
- ☐ After `create()` — notify based on status
- ☐ After `update()` — detect status change to "published"
- ☐ After `delete()` — notify all users
- ☐ Wrap each call in `try/catch` (silent failure)

EXISTING ENDPOINTS (VERIFY)

- ☐ `GET /notifications` filters by `user_id`
- ☐ `PATCH /notifications/:id/read` works
- ☐ `PATCH /notifications/read-all` works

Frontend ↔ Backend Contract Summary

LAYER	STATUS	WHAT IT DOES
Frontend — Notification Bell	✔ Done	Shows unread count badge, reads from <code>useNotificationStore.unreadCount</code>

LAYER	STATUS	WHAT IT DOES
Frontend — Notifications Page	✔ Done	Lists all notifications, mark-as-read, mark-all-as-read
Frontend — Auto-refresh after blog actions	✔ Done	Calls <code>GET /notifications</code> after every save/delete silently
Backend — <code>GET /notifications</code>	✔ Exists	Returns notifications for the authenticated user — verify it filters by <code>user_id</code>
Backend — <code>PATCH /notifications/:id/read</code>	✔ Exists	Marks one notification as read
Backend — <code>PATCH /notifications/read-all</code>	✔ Exists	Marks all user notifications as read
Backend — Blog event → notification creation	⚠ Needed	Insert notification rows in <code>BlogService</code> after create/update/delete